Fiche de présentation — TP1 SAST & SCA

VulnBank (Flask) · Bandit + SonarQube · pip-audit · Hanine Khemir — GL4 Groupe 1 · ~8 min

① Ouverture 30 s

> « Notre TP porte sur le **shift-left** : trouver et corriger les failles **pendant le développement**. On a créé une app Flask vulnérable, **VulnBank**, scannée en **SAST** (code) et **SCA** (dépendances), tout corrigé, puis re-scanné. »

Montrer l'historique Git : **baseline → fix(sast) → fix(sca)**.

② SAST — le code 3 min

« Le SAST analyse le code source. Deux outils : **Bandit** et **SonarQube**. »

- **Avant** (captures 1–2) : Sécurité **D**, 1 vulnérabilité, 3 hotspots.
- **Bandit avant** : 9 failles (injection SQL, commande, MD5, debug, TLS).

> **À dire :** « SonarQube Community **ne détecte PAS** l'injection SQL (pas d'analyse de flux). C'est **Bandit** qui la trouve → on combine plusieurs outils. »

Correction (injection SQL) : « Avant, la saisie était collée dans la requête. Après : **requête paramétrée**, la saisie ne devient jamais du SQL. » **Démo :**

```
curl -X POST localhost:5000/login \
 --data-urlencode "username=x' OR '1'='1" \
 --data "password=z"
```

« Avant : connecté. Après : **401 rejeté**. »

Après : SonarQube D → A · Bandit 9 → 0

③ SCA — les dépendances 3 min

« Le SCA compare nos dépendances aux bases de CVE. Outil : **pip-audit**. »

Avant : 7 paquets vulnérables, 44 CVE.

Exemple détaillé — Jinja2 2.10 / CVE-2019-10906 : évasion du bac à sable du moteur de templates Flask → **exécution de code**.

Pourquoi mettre à jour :

1. La faille est **publique** (numéro CVE connu de tous).
2. Le **correctif existe** (2.10.1) → plus sûr qu'un bricolage.
3. Le risque vient de la **dépendance**, pas de notre code.

Après : 44 CVE → 0 · Python 3.9 → 3.11

④ Le point honnête 30 s

> Montrer soi-même le bandeau **« Quality Gate: Failed »** : « C'est **uniquement** la couverture de tests (pas de tests dans ce TP), **pas** la sécurité. Toutes les notes sécurité = **A**. »

⑤ Conclusion 30 s

> « Bilan : **Bandit 9→0, SonarQube D→A, pip-audit 44→0**. On retient : scans automatiques avant déploiement, outils complémentaires, et le SCA tourne **en continu** (nouvelles CVE chaque semaine). »

Questions probables

Question	Réponse
Pourquoi 2 outils SAST ?	SonarQube Community rate les injections, Bandit les attrape.
Vulnerability vs Hotspot ?	Vuln = faille confirmée ; Hotspot = à vérifier par un humain.
Quality Gate rouge ?	Couverture de tests, pas la sécurité.

Glossaire — chaque terme expliqué

SAST (Static Application Security Testing) — Analyse du **code source sans l'exécuter** pour trouver des failles de programmation. Outils : Bandit, SonarQube.

SCA (Software Composition Analysis) — Vérifie si les **bibliothèques externes** (dépendances) ont des failles **déjà connues**. Analyse `requirements.txt`. Outil : pip-audit.

→ La différence : SAST = failles **qu'on a écrites** ; SCA = failles dans le code **des autres** qu'on utilise.

Shift-left — Faire la sécurité **tôt** (pendant le dev) au lieu de la fin. Corriger tôt = moins cher.

CVE — Numéro public unique d'une faille connue (ex. CVE-2019-10906), référencé dans des bases publiques.

OSV — Base de données de vulnérabilités interrogée par pip-audit.

Dépendance — Bibliothèque externe du projet (Flask, Jinja2...) avec sa version.

Injection SQL — L'attaquant insère du **code SQL** dans un champ. Correction : requête paramétrée.

Injection de commande — Idem avec des **commandes système** (os.system) exécutées sur le serveur.

XSS — Injection de **JavaScript** dans une page. Correction : auto-échappement des templates.

MD5 — Vieux hachage **cassé** pour les mots de passe. Remplacé par pbkdf2 (lent + salé).

TLS verify=False — Certificat du serveur non vérifié → attaque « man-in-the-middle ».

debug=True — Mode debug Flask : expose une console qui exécute du code. Interdit en prod.

Vulnerability (SonarQube) — Faille **confirmée**, à corriger.

Security Hotspot — Code **« à vérifier par un humain »**, pas forcément une faille.

Quality Gate — Règles à passer. Rouge ici à cause de la **couverture de tests**, pas la sécurité.

Rating A→E — Note de A (excellent) à E (mauvais). Projet : Security **D → A**.

Avant de commencer, ouvrir : le PDF (tableaux) · SonarQube localhost:9000 · un terminal pour la démo curl. · Si la démo plante → s'appuyer sur les captures du PDF.